

Sprawozdanie z Laboratorium 4 STERO

1. Konfiguracja Środowiska

Zgodnie z instrukcją, przygotowano przestrzeń roboczą dla symulacji robota TIAGo. Proces obejmował:

1. Pobranie repozytorium tiago przy użyciu komendy `vcs import --input https://raw.githubusercontent.com/RCPRGros-pkg/STERO2/refs/heads/iron/tiago_public_stero.repos src`
2. Kompilację pakietów w workspace przy użyciu `colcon build`.
3. Konfigurację zmiennych środowiskowych (`source install/setup.bash`).

2. Uruchomienie Symulacji

Symulacja została uruchomiona z pełnym stosem nawigacyjnym oraz planowaniem ruchu.

Użyta komenda:

```
ros2 launch tiago_gazebo tiago_gazebo.launch.py navigation:=True moveit:=True
is_public_sim:=True
```

W celu weryfikacji drzewa transformacji (TF) użyto narzędzia:

```
ros2 run rqt_tf_tree rqt_tf_tree --force-discover
```

Analiza Drzewa Transformacji

Na podstawie wygenerowanego grafu `tf_tree.pdf`, struktura układów współrzędnych robota TIAGo prezentuje się następująco:

1. Globalne Układy Odniesienia:

- `map`: Główny, statyczny układ współrzędnych mapy. Jest korzeniem drzewa w przypadku uruchomionej nawigacji.
- `odom`: Układ odometrii. Transformacja `map -> odom` jest dynamiczna i korygowana przez system lokalizacji, niwelując błędy dryfu odometrii.

2. Podstawa Robota:

- `base_footprint`: Rzut środka robota na płaszczyznę podłoża. Transformacja `odom -> base_footprint` jest publikowana przez kontroler podwozia (`mobile_base_controller`) na podstawie enkoderów kół.
- `base_link`: Główny człon fizyczny robota, sztywno połączony z `base_footprint`.

3. Łańcuchy Kinematyczne:

- **Tułów**: `torso_lift_link` (ruchomy w osi Z).
- **Głowa**: `head_1_link`, `head_2_link` (pan/tilt).
- **Ramię**: Seria ogniw `arm_1_link` do `arm_7_link`, zakończona punktem `wrist_ft_link` (czujnik siły/momentu).
- **Chwytek**: Zamocowany bezpośrednio za `wrist_ft_link`. Składa się z bazy (`gripper_link`) oraz ruchomych palców (`gripper_left_finger_link`, `gripper_right_finger_link`).

4. Sensory:

- **LiDAR:** `base_laser_link` - umieszczony w podstawie.
- **Kamera RGB-D:** `head_front_camera_link` (oraz ramki optyczne `_optical_frame`) - umieszczona na głowie, porusza się wraz z nią, dostarczając obraz i chmurę punktów w odniesieniu do aktualnej pozycji głowy.

Drzewo jest spójne (jeden korzeń), co oznacza, że wszystkie transformacje są poprawnie publikowane, umożliwiając działanie algorytmów planowania ruchu (MoveIt) oraz nawigacji.

Pełne drzewo transformacji zostało wyeksportowane do pliku `tf_tree.pdf`.

3. Analiza Systemu i Sensorów

Przeprowadzono weryfikację dostępnych tematów ROS 2 oraz poprawności danych z sensorów.

Komponent	Temat (Topic)	Typ Wiadomości	Status
Sterowanie	<code>/cmd_vel</code>	<code>geometry_msgs/msg/Twist</code>	Oczekujący
Odometria	<code>/mobile_base_controller/odom</code>	<code>nav_msgs/msg/Odometry</code>	OK
LiDAR	<code>/scan_raw</code>	<code>sensor_msgs/msg/LaserScan</code>	OK
Kamera RGB	<code>/head_front_camera/rgb/image_raw</code>	<code>sensor_msgs/msg/Image</code>	OK
Chmura Punktów	<code>/head_front_camera/depth_registered/points</code>	<code>sensor_msgs/msg/PointCloud2</code>	OK

4. Implementacja Algorytmu Sterowania

W celu realizacji zadania poruszania się po zadanej trajektorii (sześciokąt), zaimplementowano dedykowany węzeł ROS 2 w języku C++: `hexagon_mover.cpp`.

Kluczowe cechy implementacji:

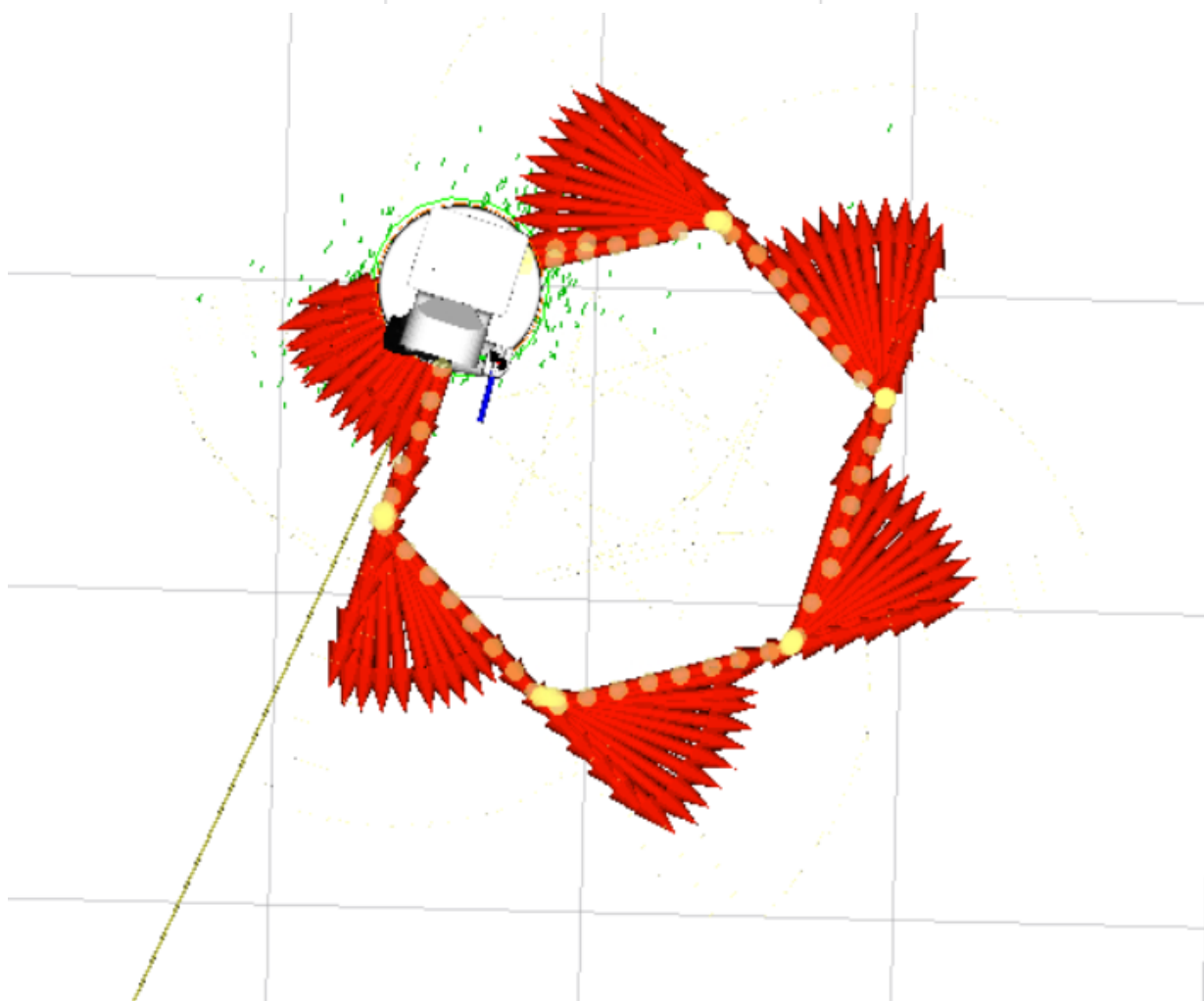
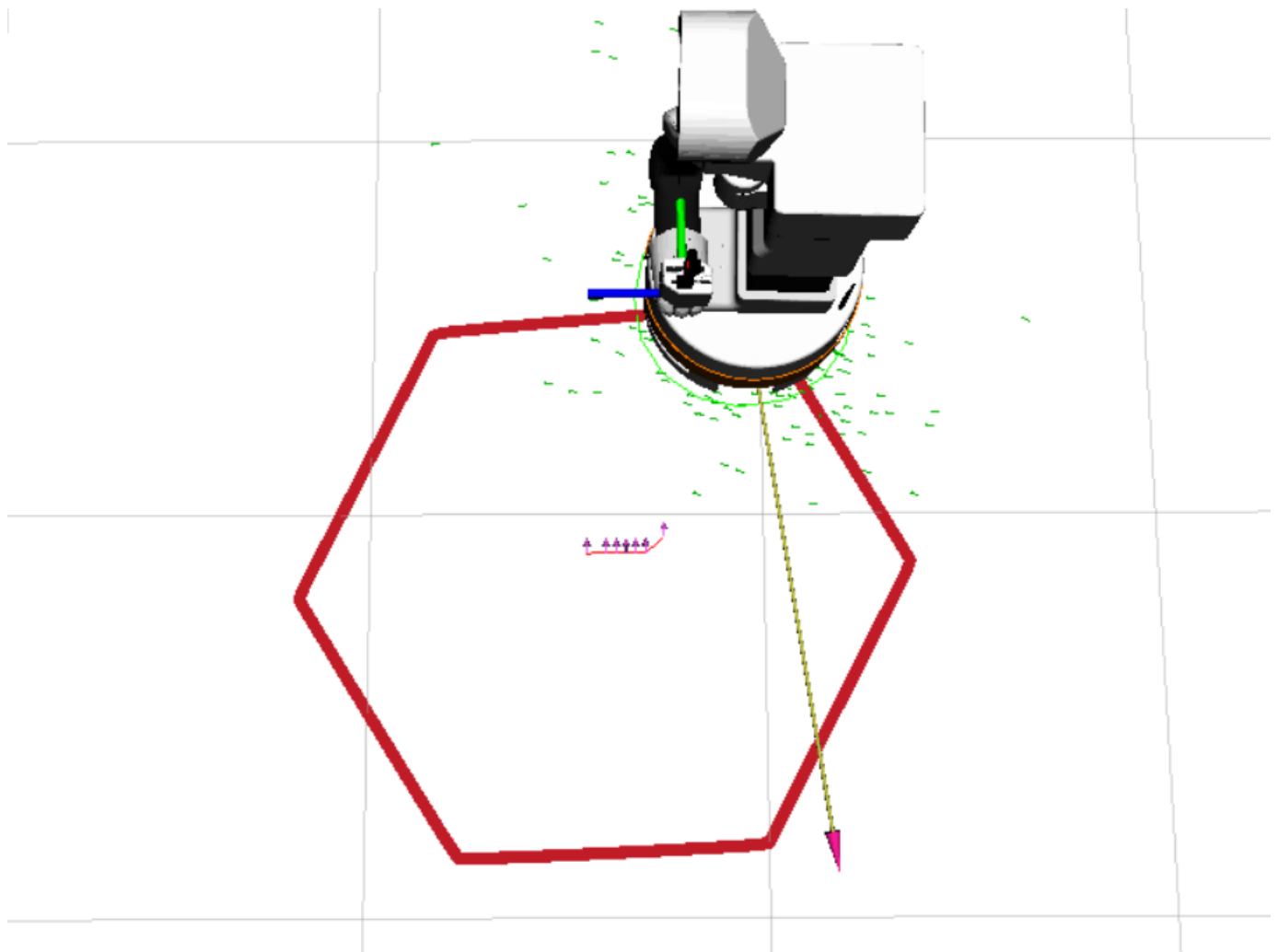
- **Typ węzła:** `rclcpp::Node` o nazwie `hexagon_mover`.
- **Komunikacja:**
 - Publikacja sterowania prędkością na temat `/cmd_vel` (`geometry_msgs/msg/Twist`).
 - Subskrypcja odometrii z `/mobile_base_controller/odom` (do logowania pozycji).
- **Algorytm:** Prosta maszyna stanów oparta na czasie.
 - Stan 0: Jazda prosto przez wyliczony czas `move_time_`.
 - Stan 1: Obrót o 60 stopni ($\pi/3$) przez czas `turn_time_`.
- **Parametry:** Prędkość liniowa `0.4 m/s`, prędkość kątowa `0.4 rad/s`.

Kod źródłowy węzła znajduje się w pliku `src/lab1_pkg/src/hexagon_mover.cpp`.

5. Weryfikacja Wizualna

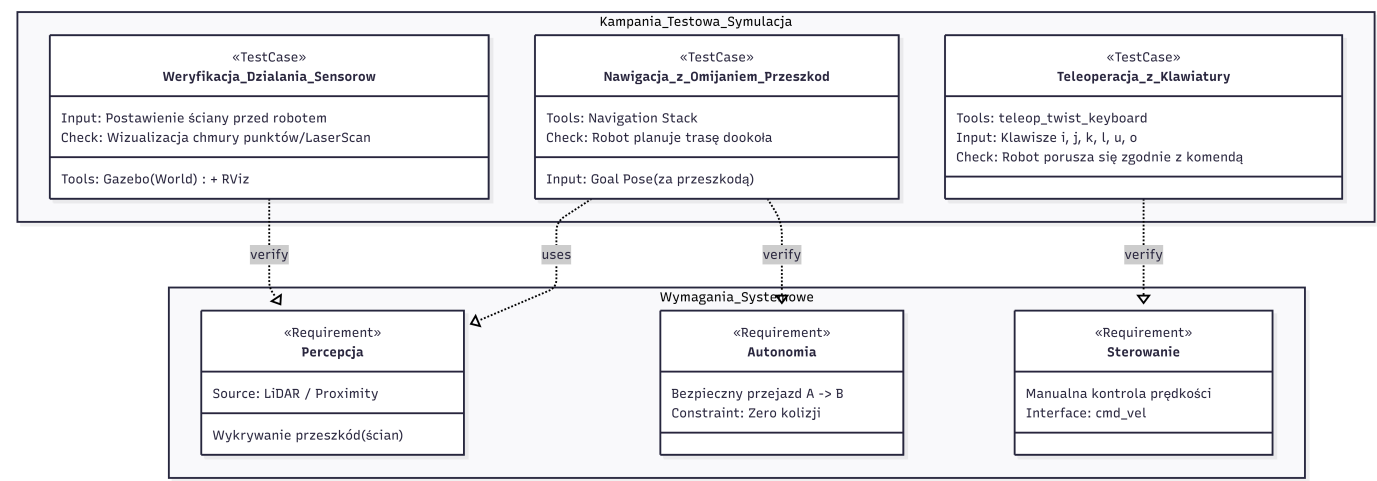
Poniżej przedstawiono zrzuty ekranu dokumentujące wykonanie zadań w środowisku symulacyjnym.

Zadanie: Hexagon



6. Plan Testów i Weryfikacja Wymagań

Struktura wymagań oraz planowane przypadki testowe zostały zamodelowane w notacji SysML.



7. Kampania Testowa

Film prezentujący przebieg wszystkich zaplanowanych testów w symulatorze.

<https://drive.google.com/file/d/1WWmyXD8RajXPDPjqZDHNNPUAdfkWYrC2/view?usp=sharing>